AIClaimsAssist

Five-Minute Demo Script | Presenter notes and on-screen actions

Demo preparation: Start Streamlit. Check that CLM-MOT-0007 still has a confirmed damage-description discrepancy and a score of 25 before using that example. Have a fallback synthetic claim ready. Do not claim live NHTSA or LLM-driven tool calling.

0:00–0:40 | Problem and goal

Show: Streamlit home screen.

“For my Week 3 project, I built AIClaimsAssist, a claims investigation assistant. Adjusters often need to review policy passages, claim notes, missing documents and inconsistencies across sources. My goal was to bring these steps into one traceable workflow and keep the final review with a human. This demo uses 20 entirely synthetic Motor and Auto claims.”

0:40–1:30 | Architecture

Show: Architecture section in project documentation.

“LangGraph orchestrates the workflow and maintains investigation state. Existing Python tools retrieve claim details, check missing documents, evaluate observations and calculate a weighted investigation-priority score. Local BM25 searches the claim’s policy with citations. An MCP server exposes VIN decoding, and LangGraph interrupts the workflow for adjuster review. This version is deterministic; it does not yet use an LLM to select tools.”

1:30–2:15 | Run investigation

Show: Select CLM-MOT-0007, click Run Investigation, open Fraud tab.

“I select a claim and run the investigation. The graph gathers evidence and policy context, evaluates discrepancies, and records its results. The score is not a probability of fraud. If this claim still shows 25, that comes from a confirmed damage-description conflict with a defined weight, rather than a model guessing a risk number.”

2:15–3:00 | Evidence and missing documents

Show: Fraud evidence, then Missing documents tab.

“The system distinguishes confirmed findings from unverified observations. An unverified related-claim observation does not automatically increase the score. In Missing documents, we can see required and conditional items and why they are outstanding. This helps the adjuster decide what needs follow-up.”

3:00–3:30 | Report and supporting tools

Show: Report tab; expand vehicle verification and policy citations.

“The report brings together the score, evidence description, missing documents, and claim summary with chronological notes. Policy citations preserve the source trail. Vehicle verification uses an MCP tool; if decoding cannot be verified, the system shows UNABLE_TO_VERIFY instead of inventing a match. Our MCP integration was validated offline with mocked responses.”

3:30–4:20 | Human review

Show: Investigation review section; select one decision and submit.

“LangGraph pauses for a human decision. The adjuster can accept findings for reporting or request more information. I’ll accept this report for reporting. The graph resumes the same investigation thread without rerunning all the tools. This is not approval or denial of the insurance claim; it only records the report-review outcome in this local session.”

4:20–5:00 | Validation and takeaway

Show: Validation table or return to dashboard.

“In final offline validation, all 83 automated tests passed, all 20 synthetic claims ran successfully, and all 20 reached the human-review checkpoint. We found zero policy-isolation failures. My key learning is that useful agentic systems need more than an LLM: they need tool boundaries, state management, explainable evidence, safe failure handling and human oversight. LLM-directed tool selection and full Hybrid RAG remain future improvements. Thank you.”

Pre-demo checklist

- Launch app.py in Streamlit and check the three tabs.
- Verify selected claim and risk score before presenting.
- Expand one policy citation and the vehicle verification result.
- Test the review decision once in a fresh investigation.
- Keep the architecture and test results available.
- Practice to remain within five minutes.